

# Defend-The-Core

---

## Annexe 2 : Sécurité & Wazuh (SIEM/XDR)

Surveillance centralisée, corrélation d'événements et réaction automatique au sein du VLAN 99

[SIEM/XDR](#) · [Active Response](#) · [MITRE ATT&CK](#) · [CIS Benchmark](#) · [OPNsense API](#)

# Table des matières

|                                                           |           |
|-----------------------------------------------------------|-----------|
| <b>1. Rôle du SIEM dans l'architecture</b>                | <b>3</b>  |
| 1.1. Corrélation vs simple collecte de logs . . . . .     | 3         |
| 1.2. Réaction automatique (active response) . . . . .     | 3         |
| 1.3. Place dans le VLAN 99 . . . . .                      | 3         |
| <b>2. Architecture Wazuh</b>                              | <b>3</b>  |
| 2.1. Flux de traitement (diagramme texte) . . . . .       | 4         |
| <b>3. Déploiement du manager</b>                          | <b>4</b>  |
| 3.1. Pré-requis . . . . .                                 | 4         |
| 3.2. Installation via dépôt officiel . . . . .            | 4         |
| 3.3. Activation du syslog entrant pour OPNsense . . . . . | 5         |
| <b>4. Déploiement des agents</b>                          | <b>5</b>  |
| 4.1. Script Linux (Debian / RPM / NixOS) . . . . .        | 6         |
| 4.2. Script Windows (PowerShell) . . . . .                | 6         |
| 4.3. Groupes d'agents . . . . .                           | 6         |
| <b>5. Règles de corrélation personnalisées</b>            | <b>7</b>  |
| 5.1. Extrait XML — règle bruteforce SSH . . . . .         | 7         |
| <b>6. Active Response (blocage automatique)</b>           | <b>8</b>  |
| 6.1. Principe et flux . . . . .                           | 8         |
| 6.2. Whitelist . . . . .                                  | 8         |
| 6.3. Déclaration dans ossec.conf . . . . .                | 8         |
| 6.4. Script block-attacker.sh . . . . .                   | 9         |
| <b>7. Security Configuration Assessment (SCA)</b>         | <b>9</b>  |
| 7.1. Politiques déployées . . . . .                       | 9         |
| 7.2. Rapport de conformité . . . . .                      | 10        |
| <b>8. Dashboards</b>                                      | <b>10</b> |
| <b>9. Choix de Wazuh vs alternatives</b>                  | <b>11</b> |

# 1. Rôle du SIEM dans l'architecture

Un pare-feu en **default-deny** et un cloisonnement rigoureux réduisent la surface d'attaque, mais ils ne détectent pas une compromission en cours. C'est le rôle du SIEM (*Security Information and Event Management*) : centraliser les journaux de l'ensemble des machines, les corrélérer et déclencher une réponse — y compris automatique — lorsqu'un comportement suspect est identifié.

## 1.1. Corrélation vs simple collecte de logs

Une simple agrégation de journaux (rsyslog vers un serveur central) ne suffit pas : elle produit des milliers de lignes brutes qu'aucun humain ne peut relire. Le SIEM ajoute une couche d'**analyse** : il applique des règles de corrélation qui croisent plusieurs événements pour en déduire une alerte exploitable.

Exemple : 5 échecs SSH isolés en une heure depuis une même IP ne sont qu'un bruit de fond. **Cinq échecs suivis d'un succès** depuis cette même IP deviennent, après corrélation, une alerte de niveau 12 : *bruteforce SSH abouti*. La corrélation transforme des signaux faibles en une détection forte.

## 1.2. Réaction automatique (active response)

Au-delà de la détection, Wazuh embarque un moteur d'**active response** : lorsqu'une règle de niveau élevé se déclenche, un script peut être exécuté automatiquement sur le manager pour, par exemple, bloquer l'IP attaquante au pare-feu via l'API OPNsense. La boucle **détection** → **décision** → **blocage** se ferme sans intervention humaine, en quelques secondes.

## 1.3. Place dans le VLAN 99

Wazuh est déployé sur le **VLAN 99 (admin/SIEM)**, la zone de confiance maximale. Toutes les VMs (VLANs 10, 20, 30) expédient leurs journaux vers l'adresse **10.10.99.10** sur les ports 514 (syslog OPNsense) et 1514 (agents Wazuh). Le flux entrant vers le SIEM est le seul autorisé depuis les zones moins fiables vers le VLAN 99 — il est strictement unidirectionnel : le SIEM écoute, il n'initie jamais de connexion vers les VLANs clients, sauf pour la remédiation via l'API OPNsense.

# 2. Architecture Wazuh

Wazuh est organisé en **trois composants** distincts, installables sur un hôte unique (déploiement monosite) ou répartis sur plusieurs nœuds (cluster). Dans le cadre de Defend-The-Core, les trois tournent sur la même VM Ubuntu 22.04 (10.10.99.10), le tout piloté par le script d'installation officiel de l'éditeur.

Tableau 1 — Composants de la stack Wazuh

| Composant | Rôle                                             | Technologie               | Port               |
|-----------|--------------------------------------------------|---------------------------|--------------------|
| Manager   | Collecte, décodage, corrélation, active response | Wazuh Server (C/OSSEC)    | 1514/udp, 1515/tcp |
| Indexer   | Stockage et indexation des alertes               | OpenSearch (fork Elastic) | 9200/tcp           |

|           |                                |                                |         |
|-----------|--------------------------------|--------------------------------|---------|
| Dashboard | Visualisation, dashboards, SCA | OpenSearch Dashboards (Kibana) | 443/tcp |
|-----------|--------------------------------|--------------------------------|---------|

### 2.1. Flux de traitement (diagramme texte)

Le diagramme ci-dessous décrit le cheminement complet d'un événement, depuis sa production sur une VM jusqu'au blocage automatique de l'attaquant :

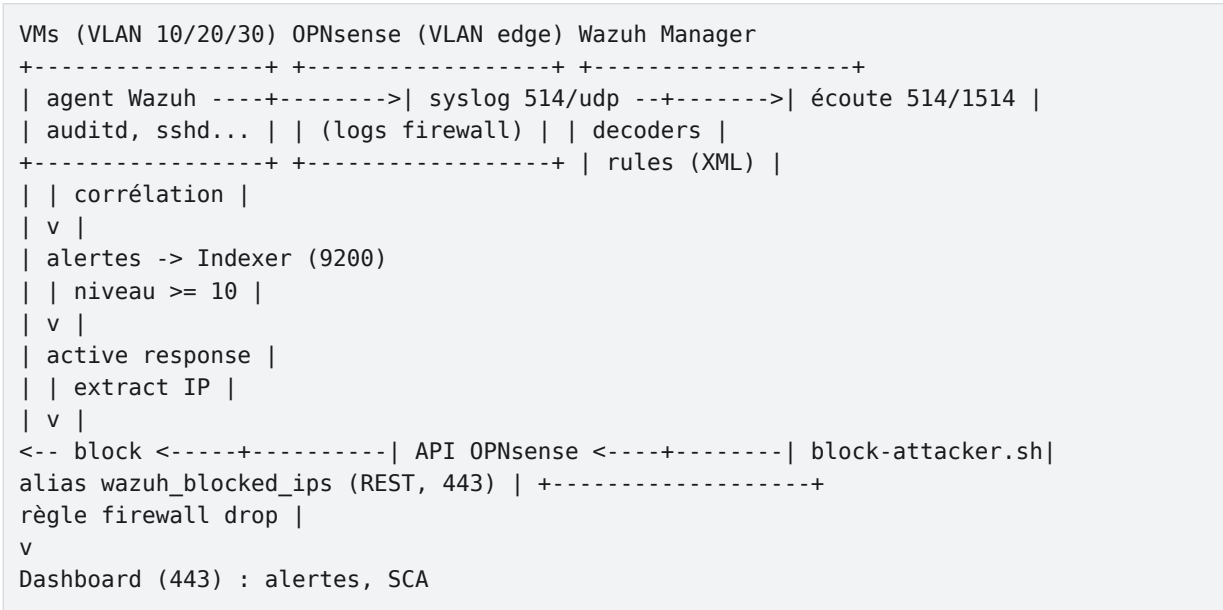


Figure 1 — Flux : logs de toutes les VMs → Wazuh (514/1514) → corrélation → alertes → active response → API OPNsense → alias dynamique → blocage.

## 3. Déploiement du manager

Le déploiement s'appuie sur le script d'installation officiel `install-wazuh-manager.sh`, fourni par l'éditeur et reposant sur le dépôt `apt` de Wazuh. Ce script installe en une passe les trois composants (Manager, Indexer, Dashboard) et génère automatiquement les mots de passe des comptes internes.

### 3.1. Pré-requis

Tableau 2 — Pré-requis matériel et système

| Ressource | Valeur                  | Justification                                         |
|-----------|-------------------------|-------------------------------------------------------|
| RAM       | 8 Go                    | Indexer OpenSearch est gourmand en mémoire (JVM heap) |
| CPU       | 4 vCPU                  | Corrélation + indexation + dashboard en parallèle     |
| Disque    | 50 Go                   | Rétention des alertes (rotation 30 jours)             |
| OS        | Ubuntu 22.04 LTS        | Supporté officiellement par le dépôt Wazuh            |
| Réseau    | VLAN 99, IP fixe .99.10 | Isolation en zone de confiance maximale               |

### 3.2. Installation via dépôt officiel

Le script ajoute la clé GPG et le dépôt Wazuh, puis installe les paquets. Extrait représentatif :

```
#!/usr/bin/env bash
# install-wazuh-manager.sh – déploiement monosite
set -euo pipefail

# 1. Ajout du dépôt apt officiel Wazuh
curl -s0 https://packages.wazuh.com/key/GPG-KEY-WAZUH
gpg --dearmor -o /usr/share/keyrings/wazuh.gpg GPG-KEY-WAZUH
echo "deb [signed-by=/usr/share/keyrings/wazuh.gpg] \
https://packages.wazuh.com/4.x/apt stable main" \
> /etc/apt/sources.list.d/wazuh.list
apt-get update

# 2. Installation des trois composants
apt-get install -y wazuh-manager wazuh-indexer wazuh-dashboard

# 3. Génération des mots de passe (admin, kibanaserver...)
/usr/share/wazuh-indexer/bin/indexer-security-init.sh
/usr/share/wazuh-manager/scripts/wazuh-passwords-tool.sh -a

systemctl enable --now wazuh-manager wazuh-indexer wazuh-dashboard
```

### 3.3. Activation du syslog entrant pour OPNsense

OPNsense ne dispose pas d'agent Wazuh : il envoie ses journaux par syslog UDP. Il faut donc configurer le manager pour qu'il écoute sur le port **514/udp** et qu'il decode le format CEF/syslog du pare-feu. Cela se déclare dans `/var/ossec/etc/ossec.conf` :

```
syslog
514
udp
10.10.99.1/32
10.10.99.10
```

Côté OPNsense, une règle d'export syslog (Interface > Logs > Settings) enverra les événements de firewall, IDS et auth vers 10.10.99.10:514.

## 4. Déploiement des agents

Chaque machine surveillée reçoit un **agent Wazuh**, processus léger qui collecte les logs locaux (auth, auditd, syslog applicatif), les pré-traite et les envoie chiffrés au manager sur le port 1514. Les agents sont répartis en **groupes** afin d'appliquer des configurations et des SCA différents selon le profil de la machine.

Tableau 3 — Agents déployés

| Machine                  | OS                  | Script de déploiement    | Groupe            |
|--------------------------|---------------------|--------------------------|-------------------|
| Ubuntu (serveur web+BDD) | Ubuntu 22.04        | deploy-agent-linux.sh    | linux, critical   |
| Win Server (AD/DNS/DHCP) | Windows Server 2022 | deploy-agent-windows.ps1 | windows, critical |

|                        |            |                          |                      |
|------------------------|------------|--------------------------|----------------------|
| Bastion NixOS          | NixOS      | deploy-agent-linux.sh    | linux, admin         |
| Win 10 (poste employé) | Windows 10 | deploy-agent-windows.ps1 | windows, workstation |
| Kali (démon attaque)   | Kali Linux | deploy-agent-linux.sh    | linux, workstation   |

## 4.1. Script Linux (Debian / RPM / NixOS)

Le script `deploy-agent-linux.sh` détecte la famille de distribution et branche le bon gestionnaire de paquets. NixOS, qui ne supporte pas les binaires précompilés Wazuh, est géré via le module Nix déclaratif `services.wazuh-agent` dans `configuration.nix`.

```
#!/usr/bin/env bash
# deploy-agent-linux.sh – installe l'agent Wazuh sur Linux
MANAGER_IP="10.10.99.10"
GROUP="${1:-linux}"

if grep -qi nixos /etc/os-release; then
# NixOS : configuration déclarative (immuable)
echo 'services.wazuh-agent.enable = true;' >> /etc/nixos/wazuh.nix
echo 'services.wazuh-agent.settings.client.address = "'$MANAGER_IP'";' \
>> /etc/nixos/wazuh.nix
nixos-rebuild switch
elif grep -qi debian /etc/os-release; then
curl -so wazuh-agent.deb https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4.7.0-1_amd64.deb
WAZUH_MANAGER="$MANAGER_IP" WAZUH_GROUP="$GROUP" dpkg -i wazuh-agent.deb
systemctl enable --now wazuh-agent
else # famille RPM (RHEL/Fedora/CentOS)
rpm -i https://packages.wazuh.com/4.x/yum/wazuh-agent-4.7.0-1.x86_64.rpm
sed -i "s/MANAGER_IP/$MANAGER_IP/" /var/ossec/etc/ossec.conf
systemctl enable --now wazuh-agent
fi
```

## 4.2. Script Windows (PowerShell)

```
# deploy-agent-windows.ps1 – agent Wazuh sur Windows
$ManagerIP = "10.10.99.10"
$Group = if ($args[0]) { $args[0] } else { "windows" }

Invoke-WebRequest -Uri "https://packages.wazuh.com/4.x/windows/wazuh-agent-4.7.0-1.msi" `
-OutFile "$env:TEMP\wazuh-agent.msi"

msiexec /i "$env:TEMP\wazuh-agent.msi" `
/q WAZUH_MANAGER="$ManagerIP" WAZUH_REGISTRATION_SERVER="$ManagerIP" `
WAZUH_AGENT_GROUP="$Group"

Start-Service WazuhSvc
Set-Service WazuhSvc -StartupType Automatic
```

## 4.3. Groupes d'agents

Les groupes permettent de pousser des `shared.conf` et des politiques SCA différenciés. Cinq groupes sont définis :

- **linux** — configuration de base pour tous les agents Linux (auditd, syslog).
- **windows** — collecte EventLog, Sysmon, Defender sur les postes Windows.
- **critical** — machines du VLAN 30 (serveurs métier) : SCA CIS strict, alertes niveau +2.
- **admin** — bastion NixOS : surveillance renforcée des commandes sudo et des modifications système.
- **workstation** — postes utilisateur (VLAN 10) : détection d'exécution anormale, USB, etc.

## 5. Règles de corrélation personnalisées

Wazuh embarque plusieurs milliers de règles OSSEC couvrant les usages courants. Pour le projet Defend-The-Core, des règles **custom** (IDs 100100 à 100900) sont ajoutées dans `/var/ossec/etc/rules/local_rules.xml`. Elles ciblent les scénarios d'attaque spécifiques au lab et sont annotées avec les techniques **MITRE ATT&CK**; correspondantes.

Tableau 4 — Règles de corrélation personnalisées

| ID     | Niveau | Déclencheur                                                 | MITRE ATT&CK;                   |
|--------|--------|-------------------------------------------------------------|---------------------------------|
| 100100 | 10     | Bruteforce SSH : 5 échecs puis 1 succès / 1 min             | T1110 Brute Force               |
| 100200 | 7      | Scan Nmap détecté (auditd connect() sur plage)              | T1046 Network Service Discovery |
| 100300 | 12     | Mouvement latéral : SSH vers un autre VLAN depuis un poste  | T1021 Remote Services           |
| 100400 | 11     | Modification de fichier de config système (sshd, sudoers)   | T1543.002 Persistence           |
| 100500 | 12     | Persistence / backdoor : cron atypique ou clé SSH ajoutée   | T1053.003 Cron                  |
| 100600 | 10     | Désactivation d'UFW ou de auditd                            | T1562 Impair Defenses           |
| 100700 | 9      | Exécution de reverse shell (bash -i >& /dev/tcp)            | T1059 Command Scripting         |
| 100800 | 11     | Création de compte utilisateur en dehors des heures ouvrées | T1136 Create Account            |
| 100900 | 13     | Plusieurs règles critiques (100300+100500) en 5 min         | TA0001 Initial Access           |

### 5.1. Extrait XML — règle bruteforce SSH

La règle 100100 illustre la corrélation : elle se déclenche quand la règle native OSSEC 5712 (authentification SSH réussie) suit, dans un délai d'une minute, au moins cinq occurrences de la règle 5710 (échec SSH) depuis la même IP source.

```
5712
authentication_success

SSH bruteforce réussi: 5 échecs puis un succès

T1110
```

```
alert_by_email
```

Le niveau 10 (> 7) déclenche automatiquement l'active response (voir section 6) : l'IP source est poussée dans l'alias OPNsense et bloquée sans intervention manuelle.

## 6. Active Response (blocage automatique)

L'active response est le mécanisme qui transforme Wazuh d'un simple **détecteur** en un **réacteur**. Lorsqu'une alerte atteint un niveau suffisant, le manager exécute un script — ici `block-attacker.sh` — qui extrait l'IP source de l'alerte et demande au pare-feu OPNsense de la bloquer via son API REST.

### 6.1. Principe et flux

Le flux complet, exécuté en quelques secondes :

- **Alerte** — une règle de niveau  $\geq 10$  se déclenche (ex. 100100 bruteforce SSH).
- **Extraction IP** — le moteur d'active response parse l'alerte et isole le champ `srcip`.
- **Appel API OPNsense** — `block-attacker.sh` fait un POST sur `/api/firewall/alias_utility/add/wazuh_blocked_ips`.
- **Alias dynamique** — l'IP est ajoutée à l'alias `wazuh_blocked_ips` côté OPNsense.
- **Règle de blocage** — une règle `block` sur l'alias rejette tout trafic de ces IP sur toutes les interfaces.
- **Timeout** — après 1 h, l'IP est retirée de l'alias (débloquée) via un appel symétrique.

### 6.2. Whitelist

Pour éviter de se bloquer soi-même, une **liste blanche** d'IP non-bloquables est définie dans `ossec.conf`. Elle inclut au minimum :

- le **bastion** (10.10.99.20) — point d'administration légitime,
- le **serveur Wazuh** lui-même (10.10.99.10) — pour éviter une auto-exclusion,
- la **passerelle OPNsense** (10.10.99.1) — source de nombreux logs légitimes.

### 6.3. Déclaration dans ossec.conf

La commande d'active response est déclarée puis référencée sur les règles de niveau  $\geq 10$ , avec un timeout de 3600 secondes (1 heure) :

```
block-attacker
block-attacker.sh
yes
```



```
block-attacker
local
100100,100300,100500,100900
3600
30,60,120

10.10.99.20
10.10.99.10
10.10.99.1
```

## 6.4. Script block-attacker.sh

Le script reçoit l'IP en variable d'environnement ALERTSRCIP et appelle l'API OPNsense avec une clé d'API dédiée (droit minimal : gestion de l'alias uniquement).

```
#!/usr/bin/env bash
# block-attacker.sh – active response Wazuh -> OPNsense API
OPNSENSE_API="https://10.10.99.1/api"
API_KEY="${OPN_API_KEY}" # secret, hors du dépôt
API_SECRET="${OPN_API_SECRET}"
ALIAS="wazuh_blocked_ips"

ACTION="$1" # add | delete
SRCIP="${ALERTSRCIP:-}"
[ -z "$SRCIP" ] && exit 0

case "$ACTION" in
add)
curl -sk -u "$API_KEY:$API_SECRET" -X POST \
"$OPNSENSE_API/firewall/alias_utility/add/$ALIAS" \
-d "{\"address\":\"$SRCIP\"}"
;;
delete)
curl -sk -u "$API_KEY:$API_SECRET" -X POST \
"$OPNSENSE_API/firewall/alias_utility/delete/$ALIAS" \
-d "{\"address\":\"$SRCIP\"}"
;;
esac
# Applique la nouvelle config firewall
curl -sk -u "$API_KEY:$API_SECRET" -X POST "$OPNSENSE_API/firewall/apply"
```

## 7. Security Configuration Assessment (SCA)

Le SCA est un module Wazuh qui évalue **en continu** la conformité des machines à un référentiel de durcissement — ici les **CIS Benchmarks**. Contrairement à un scanner ponctuel (Lynis, OpenSCAP), le SCA s'exécute périodiquement sur chaque agent et remonte un score de conformité ainsi que la liste des contrôles échoués, directement dans le dashboard.

### 7.1. Politiques déployées

Tableau 5 — Politiques SCA

| Policy                  | Cible           | Contrôles (extrait)                                                    |
|-------------------------|-----------------|------------------------------------------------------------------------|
| CIS Ubuntu 22.04        | Serveur Linux   | SSH root interdit, systemctl hardening, UFW, auditd, permissions /etc  |
| CIS Windows Server 2022 | Contrôleur AD   | SMBv1 off, mot de passe $\geq 14$ , audit policy, NLA RDP              |
| CIS Windows 10          | Postes employés | Defender on, BitLocker, AppLocker, restriction USB                     |
| DTC-bastion (custom)    | Bastion NixOS   | Immuabilité (nixos-rebuild), FIDO2, ProxyJump only, pas de root direct |

## 7.2. Rapport de conformité

Chaque exécution produit un score (0–100 %) par agent et par policy. Une alerte est levée si le score d'une machine critique chute sous 85 %, ce qui permet de détecter une **dérive de configuration** — par exemple une règle UFW supprimée manuellement ou un service durci désactivé. Le dashboard « SCA » agrège les scores par groupe et par policy, et expose l'historique des contrôles échoués pour audit.

**Lien avec le bastion NixOS** : sur le bastion, le SCA vérifie que la configuration reste *identique* à l'état déclaré dans Git. Toute modification non versionnée est signalée comme non-conforme — l'immuabilité garantit qu'un `nixos-rebuild switch` la réécrasera.

## 8. Dashboards

Le dashboard Wazuh (OpenSearch Dashboards) expose des visualisations prêtes à l'emploi agrégeant les alertes, les active responses et l'état des agents. Huit panels personnalisés sont configurés pour le projet :

Tableau 6 — Panels de dashboard

| Panel            | Visualisation                                         | Usage                                                  |
|------------------|-------------------------------------------------------|--------------------------------------------------------|
| alerts-level     | Histogramme empilé par niveau (0–15)                  | Vue d'ensemble du bruit et des alertes critiques       |
| bruteforce-ssh   | Compteur + top IP source                              | Suivi des tentatives SSH (règle 100100)                |
| top-source-ips   | Bar chart horizontal des IP les plus bruyantes        | Repérer les sources récurrentes à bloquer              |
| active-response  | Timeline des blocages auto                            | Vérifier que block-attacker.sh s'exécute               |
| firewall-blocks  | Compteur des drops OPNsense (alias wazuh_blocked_ips) | Mesurer l'effet du blocage côté firewall               |
| agents-status    | Tableau active/disconnected par groupe                | Disponibilité des agents (linux, windows, critical...) |
| lateral-movement | Graphe source → destination des connexions inter-VLAN | Détecter un mouvement latéral (règle 100300)           |
| config-changes   | Liste temporelle des modifs de fichiers de config     | Suivi des règles 100400/100500 (persistance)           |

Ces panels sont exportés au format `.ndjson` (Saved Objects) et versionnés dans le dépôt Git, ce qui permet de les réimporter sur toute instance Wazuh fraîche et de rejouer les démonstrations d'entretien de façon reproductible.

## 9. Choix de Wazuh vs alternatives

Wazuh a été retenu face à deux alternatives courantes : un stack **Elastic pur** (Elasticsearch + Kibana + Logstash, sans le couche OSSEC) et **Splunk** (solution commerciale). Le tableau ci-dessous résume la comparaison sur les critères décisifs pour une PME.

Tableau 7 — Comparatif Wazuh / Elastic pur / Splunk

| Critère                    | Wazuh                  | Elastic pur           | Splunk                         |
|----------------------------|------------------------|-----------------------|--------------------------------|
| Coût                       | Open source (gratuit)  | Open source (basique) | Commercial, cher à l'ingestion |
| Active response native     | Oui (scripts OSSEC)    | Non (à développer)    | Partiel (via SOAR payant)      |
| SCA (CIS en continu)       | Oui, intégré           | Non (scanner externe) | Non (addon séparé)             |
| Règles OSSEC prêtes        | + 3000 incluses        | Aucune (à écrire)     | Addons ES, payants             |
| Agents multi-OS            | Linux/Win/macOS        | Beats (limité)        | Universal Forwarder            |
| Communauté / maturité      | Fork OSSEC, très actif | Très mature           | Très mature                    |
| Intégration firewall (API) | Via active response    | À scripter            | Via SOAR                       |
| Courbe d'apprentissage     | Moyenne                | Élevée (pile ELK)     | Faible (mais payant)           |

Le facteur décisif est l'**active response native** couplée au SCA : seul Wazuh offre, dans une même brique open source, la détection (règles OSSEC), la réaction (blocage auto via API OPNsense) **et** le contrôle continu du durcissement (CIS). Un stack Elastic pur aurait exigé de réécrire ces trois couches ; Splunk les fournit mais à un coût incompatible avec un budget PME.

**Synthèse** : Wazuh est le seul composant du projet qui ferme entièrement la boucle *détecter* → *décider* → *bloquer* sans intervention humaine ni licence commerciale, tout en s'intégrant nativement à l'API OPNsense via l'alias `wazuh_blocked_ips`.